Innovation
Concept Assessment

ICT30016

Technology Innovation Project

Submission date: 23rd October 2026
Supervisor name: Dr Kelvin Yong

Student Name	Student Id
Jane Yan Zhen YU	104395810
Yan Han WONG	104392138
Bruce Gee Yick WONG	104401843
Neville Zi Yang	104400992

Executive Summary

Our client requires a lightweight, real-time intrusion detection system (IDS) for a campus network, prioritizing low false alarms and an interactive dashboard. The live campus network remains out of scope, so all testing is strictly confined to a controlled lab environment using synthetic attacks and public datasets.

To meet these requirements, this report proposes an evidence-based, dual-track innovation concept.

Primary Design: Hybrid Three-Layer Detection

Our primary design employs a hybrid architecture to deliver immediate value and reliability. It combines Suricata rules for known signatures, a custom statistical layer for volumetric floods, and an unsupervised Isolation Forest model to catch abnormal flow sizes. In early prototype testing, the rules and statistical layer detected lab-generated attacks, and the dashboard showed these alerts almost immediately after we shortened the Suricata flow timeout. However, during the testing of our early prototype, we found that the unsupervised model, which evaluates one network flow at a time, detected none of the DDoS or port scan flows in the CIC-IDS2017 benchmark.

Secondary Design: Four-Layer Detection with a Supervised Random Forest

To address this specific failure, we propose our secondary design, which acts as the full realization of our machine learning backup plan. It keeps all three layers of the primary design and adds a fourth layer: a supervised Random Forest classifier that looks at the behaviour of one source IP over a 10-second window instead of one flow. This allows the system to learn the actual behaviour of volumetric attacks from labelled lab data rather than relying only on manual statistical thresholds, while the existing layers keep catching what they already catch.

Both designs are checked against the same client requirements. They must be lightweight and run on a single commodity PC, show alerts in real time on an interactive dashboard, keep false alarms low, and be tested only in a controlled lab. Design 1 meets these requirements with three layers that need no cloud service or labelled data. Design 2 uses the same capture pipeline and dashboard and runs the Random Forest next to the three existing layers, so it adds a supervised model without adding new infrastructure. If the Random Forest layer does not improve detection, we switch it off, and the client still receives Design 1 as a working system.

By developing these two designs in parallel, we provide the client with a practical, immediate solution today, alongside an extended version that adds a data-driven layer and that we will test against it on the same data.

Primary Design: Hybrid Three-Layer Detection

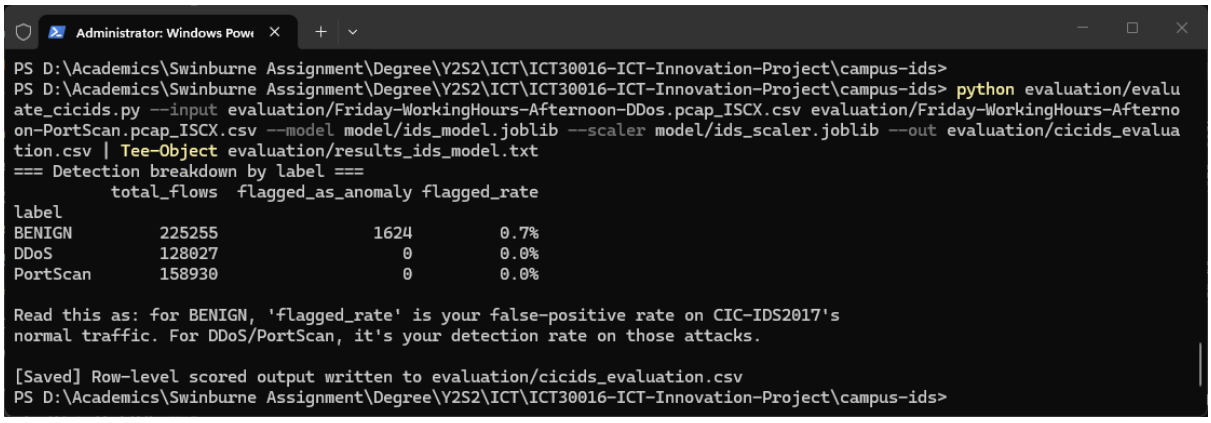
Design Concept

Design 1 is a low-cost intrusion detection system (IDS) for a small campus-style network. It will find attacks with three layers and show every alert on one real-time dashboard.

The three layers are: (1) Suricata rules for known patterns, (2) a statistical detector that counts connections per source IP to find port scans and floods, and (3) an Isolation Forest model [1] to find flows with unusual size or volume. The dashboard will show all alerts in one table with a Source column (Rule, Stats or ML), a severity level, charts and CSV export.

The design has three aims:

- Find common lab attacks (ping, port scan, SYN flood) with at least one of the three layers.
- Show each alert on the dashboard in under 500ms.
- Run on one PC with free tools, so a small team can set it up quickly.



Our prototype testing demonstrates exactly why three layers are necessary. When evaluated on the Friday files of the CIC-IDS2017 dataset [2], our baseline-trained Isolation Forest model maintained a low false-positive rate of 0.7% on normal traffic. However, it still detected none of the DDoS flows (0 of 128,027) or port scan flows (0 of 158,930) [2]. It looks at one flow at a time, so it cannot see that many flows from one source form a scan. Furthermore, machine learning models often struggle in real-world network deployments and generate false alarms when faced with traffic they were not explicitly trained on [3]. So, we will keep the model for size and volume outliers, and use rules and statistics for scans and floods. Our core innovation lies in this architectural integration and the real-time dashboard. The system is highly lightweight, designed for rapid deployment on a single commodity PC without relying on external cloud services or paid API subscriptions.

The design builds on existing approaches. Rule-based tools such as Suricata and Snort are fast and accurate for known attacks, but they miss new patterns. ML-based systems can find new patterns, but they often give many false alerts and need good training data [3]. Many systems use only one of these methods. Our design keeps the Suricata rules, adds a simple statistical layer for scans and floods, and uses ML only for size and volume outliers. The dashboard is planned to add the ability to read alerts, filter, and export them.

Methodology

Our process methodology to achieve the primary design follows a Hybrid Agile framework. The fixed unit deadlines are managed using a Waterfall approach, while the core technical development is executed through iterative Agile sprints. The live campus network remains out of scope, so all building and testing occur in a controlled lab environment.

1. Capture normal traffic with Suricata and turn the flow records into features.
2. Train the Isolation Forest [1], [4] on the normal traffic.
3. Add Suricata rules and the statistical detector for scans and floods.
4. Check the system with the datasets CIC-IDS2017 [2] and NSL-KDD [5]. We will use them only as benchmarks.
5. Test with live attacks (ping, port scan, SYN flood) from a Kali Linux VM against a Windows host that runs Suricata.
6. After each test, decide what to tune or change, and write the decision in the project log.

This method completely aligns with the client's limits. All tests stay in the lab, the tools are lightweight and free, and every integration sprint continuously verifies the 500ms UI alert target.

Specification

Item	Detail
Input	Suricata flow records (eve.json) and raw packets
Unit of analysis	Single flow (Isolation Forest); per-IP sliding window (Statistical layer)
Features	Flow duration, packet counts, byte rates (extracted from eve.json)
Model	Isolation Forest (scikit-learn), Statistical Thresholds, and Suricata Rules
Output classes	Normal, Anomaly (ML), Ping, Port scan, Flood (Rule/Stats)
Language and libraries	Python, scikit-learn [4], pandas
Capture	Suricata flow logging to eve.json
Dashboard	Flask and Flask-SocketIO dashboard with dynamic UI, filters, and CSV export
Lab environment	VMware: Kali Linux attacker VM and a target host running Suricata, on a bridged network
Training data	Own unlabelled baseline lab captures (unsupervised learning)
Evaluation data	Held-out lab runs, CIC-IDS2017, and NSL-KDD (benchmark only)
Code management	Git and GitHub feature branch with review before merging
Detection delay	Target: under 500 ms. In lab demonstrations, alerts appeared near-instantly after we shortened the Suricata TCP flow timeout to 1 second. Formal measurement is planned during testing.

Design 1 aims for minimal detection delay (target under 500 ms) by evaluating single flows and using fast rule and statistical matching. To get quick feedback during demonstrations, we shorten the Suricata TCP flow timeout, so flow records are written quickly, and alerts appear on the dashboard almost immediately.

Milestones and Risks

Week	Milestone	Output
7	Merge hybrid detection into main; finish dashboard redesign	Working main branch
8	Test false alarms on normal browsing; tune limits; re-check CIC-IDS2017 results	Final settings and benchmark results
9	Live attack tests from Kali VM; Stand up 2 (2 Nov)	Test results; progress, next steps and blockers
10	Draft testing and evaluation report; clean GitHub repository	Report draft, clean repository
11	Demo; fix any problems found	Live attack demo
12	Submit testing and evaluation report and GitHub repository; peer assessment	Final report and repository

No	Risk	Likelihood	Impact	Mitigation
1	ML layer gives many false alerts on normal traffic	High	Medium	Let users filter by Source; treat ML alerts as low priority; state it as a known limit
2	Statistical rules give false alarms on busy normal traffic	Medium	Medium	Adjustable limits and trusted IPs; test normal browsing before the demo
3	Alerts appear too late because of Suricata flow timeouts	Medium	High	Use short timeouts for the demo; record the settings used in each test
4	Network or IP changes break detection	Medium	Medium	Automatic own-IP mode; warning banner; start script
5	Too much work depends on one team member	Medium	High	Use pulls requests; write a README and lab guide; share tasks

Secondary Design: Four-Layer Detection with a Supervised Random Forest

Design Concept

In early prototype testing, the Isolation Forest in Design 1 detected none of the DDoS flows (0 of 128,027) or port scan flows (0 of 158,930) in the Friday files of CIC-IDS2017 [2], while flagging 0.7% of the normal flows. This is likely because it evaluates each flow on its own, which makes scans and floods look normal. In Design 1, these attacks are caught only by Suricata rules and statistical checks that rely on hand-set thresholds, so the machine learning layer adds little for them. Missing these attacks is a high-impact risk, because port scans often come before an intrusion and floods can take a service offline. In our lab tests, these two layers already caught almost all of our attacks, so the new layer must show that it adds something beyond them, such as learning the limits from data instead of setting them by hand

Design 2 keeps all three layers of Design 1 and adds a Random Forest layer to address this blind spot in three ways:

- detect scans and floods by looking at the behaviour of one source IP over a 10-second window, instead of one flow
- learn the difference between normal and attack behaviour from labelled data with a supervised Random Forest classifier [6], instead of relying on hand-set thresholds
- be tested directly against each layer of Design 1 on the same data, so the comparison is fair

The Suricata capture pipeline and the dashboard stay the same. The Rule, Statistical, and Isolation Forest layers keep running, and the Random Forest alerts appear as a fourth source on the same dashboard.

Detecting scans by tracking a source's behaviour across connections is a well-known idea [7]. Random Forest was among the classifiers evaluated when CIC-IDS2017 was introduced and was reported as the best performer in terms of accuracy and execution time [2]. Our innovation is not a new algorithm. It is the integration as we add a per-IP window layer with a supervised model into the same lightweight system as Design 1 and test it on the same data as the other layers, so the final choice is based on evidence.

Design 2 is Design 1 plus one extra layer. The differences are:

	Design 1	**Design 2**
Learning	Unsupervised (no labels)	Design 1 layers plus a supervised layer (needs labels)
Unit of analysis	One flow (ML layer); per-IP counts with fixed thresholds (statistical layer)	Same as Design 1, plus one source IP per 10 s window (Random Forest layer)
Strength	Can flag traffic it has never seen	Keeps everything Design 1 detects and adds scans and floods learned from labelled data, with no hand-set limits
Weakness	The Isolation Forest layer misses scans and floods	The Random Forest layer is weaker on new attack types and needs labelled data

Methodology

Our innovation process stems directly from our testing evidence. After finding that Design 1's Isolation Forest could not detect port scans and floods, we propose adding a fourth layer that learns these behaviours directly from data. The live campus network remains out of scope.

1. Capture normal traffic with Suricata on our own network to use as the baseline.
2. Generate live attacks (nmap port scans, hping3 SYN floods) from a Kali Linux VM and automatically label the traffic windows using attack time logs.
3. Read the Suricata eve.json flow records with a Python module and group them by source IP into 10-second windows to extract features.
4. Split the labelled windows into training and test sets strictly by attack run and time, not randomly, and apply class weights.
5. Train the supervised model on this lab data to recognize the attack patterns.
6. Check the system with leave-one-day-out cross-validation on our labelled Kali runs, so every window is predicted by a model that never saw that day. We will also try the Friday DDoS and PortScan files of CIC-IDS2017 as an outside benchmark, but only if we can get the version with source IP and timestamp columns.
7. Compare precision, recall, F1 and false-positive rate of the Random Forest layer with each layer of Design 1, and of the layers combined, on the exact same windows.

We will treat the Random Forest layer as worth keeping only if, on held-out lab runs, it detects clearly more scan and flood windows than the Isolation Forest layer, and, next to the statistical layer, it adds attacks that layer misses without adding more false alarms or gives similar results without hand-set limits. If CIC-IDS2017 can be used, the same checks apply there. We will also record how much hand tuning each layer needs when the attack speed changes. These criteria are checked at the Week 8 decision point.

Specification

Item	Detail
Input	Suricata flow records (EVE JSON)
Unit of analysis	One source IP per 10-second window (Random Forest layer); the other layers use the units of Design 1
Features	Connection count, distinct destination ports, reset ratio, average packets per flow, total bytes
Model	Isolation Forest, statistical thresholds and Suricata rules (as Design 1), plus a class-weighted Random Forest (scikit-learn)
Output classes	Design 1 outputs, plus Normal or attack from the Random Forest (port scan and SYN flood are both labelled as attack in the prototype)
Language and libraries	Python, scikit-learn [4], pandas
Capture	Suricata flow logging to EVE JSON
Dashboard	Four alert sources on one dashboard: Rule, Statistical, ML (Isolation Forest) and Random Forest, each with its own colour and filter
Lab environment	VMware: Kali Linux attacker VM and a target host running Suricata, on a bridged network
Training data	Own baseline captures plus labelled Kali VM attack runs
Evaluation data	Held-out lab runs and CIC-IDS2017 (benchmark only)
Code management	Git and GitHub feature branch with review before merging
Detection delay	Up to one window (10 s) plus about 5 s of waiting for late flow records, so about 15 s in total. Rule and statistical alerts are still immediate.

The Random Forest layer increases the detection delay of its own alerts, because it waits for the 10-second window to close and the flow timeout to trigger. This is an acceptable trade-off, because the other layers still alert immediately and the new layer captures cross-flow behaviour for prolonged attacks like scans and floods.

Milestones and Risks

Week	Milestone
Tuition-free week (12 Oct)	Built window feature extractor and first Random Forest (prototype) to run next to the Design 1 layers
7 (19 Oct)	Collect labelled Kali attack data; train initial Random Forest model.
8 (26 Oct)	Tune model; evaluate on lab data; verify CIC-IDS2017 dataset. Decision point: review lab evaluation metrics and initial CIC-IDS2017 checks to confirm whether to keep the Random Forest layer.
9 (2 Nov)	Present initial results; begin dashboard integration.
10 (9 Nov)	Complete dashboard integration; finalize CIC-IDS2017 comparison if the Traffic Labelling version is available.
11 (16 Nov)	Integrate the Random Forest layer into the final demo, next to the three Design 1 layers.
12 (23 Nov)	Document results in final testing and evaluation report.

No	Risk	Likelihood	Impact	Mitigation
1	Model overfits to our Kali tools, because we only have one attacker VM and one other device	High	High	Vary attack tools and speeds before Week 8; keep Design 1 as the fallback
2	Severe class imbalance between attack and normal windows.	High	Medium	Apply class weights; generate diverse attack speeds.
3	Model fails on unseen attacks (supervised learning limit).	High	Medium	Document limitation, as ML systems often struggle with traffic, they were not explicitly trained on [3]; retain Design 1 to catch unknown traffic.
4	The CIC-IDS2017 files we downloaded have no source IP or timestamp, so windows cannot be built	High	Medium	Use the Traffic Labelling version of the Friday files; fall back to lab-only data if it cannot be used
5	Heavy normal traffic (e.g., video) triggers false alarms.	Medium	Medium	Expand training baseline; implement trusted-IP whitelist.
6	Data collection and labelling exceed timeline.	Medium	Medium	Automate labelling via logs; restrict scope to two attack types.
7	The Random Forest layer performs poorly or development is delayed.	Medium	Medium	Switch the layer off; the three Design 1 layers keep running as Design 1
8	Very slow attacks are missed (a SYN flood of 10 packets per second was missed by the statistical layer and found by the Random Forest in 2 of 3 windows; nmap -T2 scans were caught).	Medium	Medium	State it as a known limit; try a longer window as future work
9	The Random Forest missed all full-speed hping3 floods on the held-out day (0 of 5 windows), while the statistical layer caught them.	Medium	Medium	Test several flood speeds; keep the statistical flood rule running next to the model
10	Model alerts arrive about 15 s late (10 s window plus a short wait for late flow records)	High	Low	Rule and statistical alerts stay immediate; the Random Forest is a second opinion

Design Justification

Our dual-design approach is heavily driven by empirical evidence and practical engineering constraints. Design 1 is a **working three-layer system**, and Design 2 **adds a supervised Random Forest layer** on top of it. This way the client always receives a reliable product, while we investigate a data-driven solution to its weaknesses.

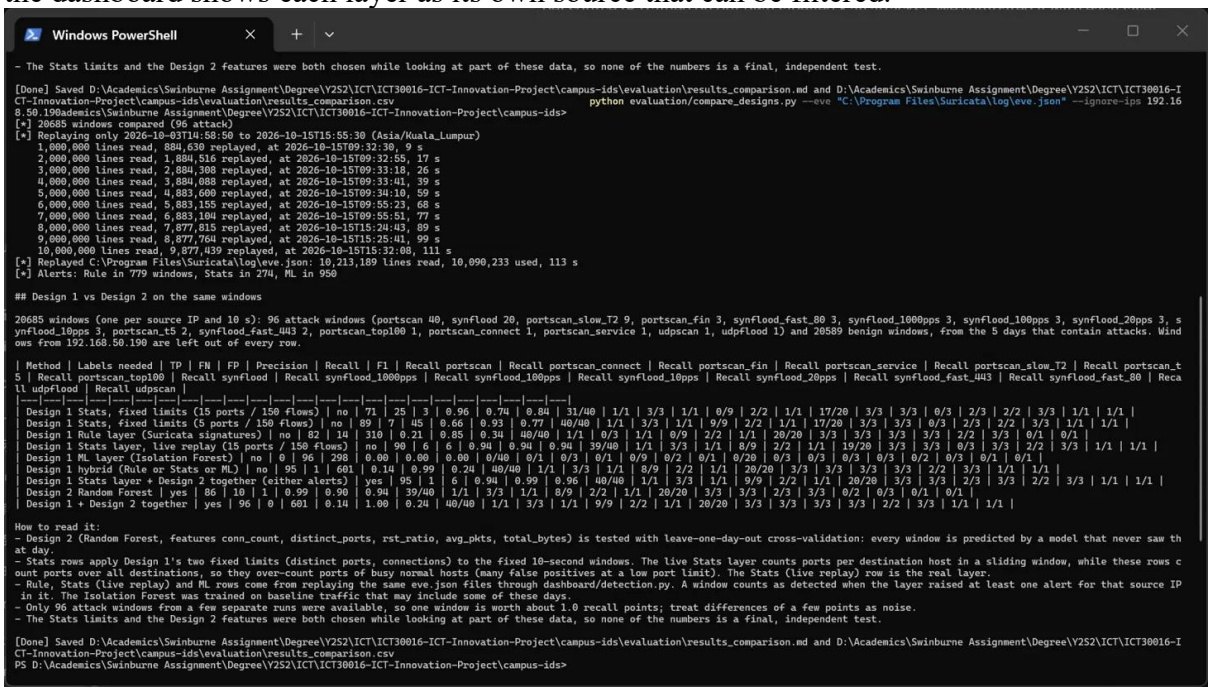
Other Published Methods and Our Innovation

Current industry standards heavily rely on rule-based systems like Snort and Suricata. While highly accurate for known threats, they have limited capability against novel zero-day attacks. Conversely, anomaly detection using unsupervised machine learning, such as Isolation Forest [1], promises to catch novel threats but historically suffers from high false-positive rates. Supervised models like Random Forest [6] are widely published for their high accuracy on datasets like CIC-IDS2017 [2]. However, as noted by Sommer and Paxson [3], machine learning systems often struggle in real-world network deployments and generate false alarms when faced with traffic they were not explicitly trained on. Our innovation does not lie in inventing a new mathematical algorithm. Instead, we add per-IP window features, a well-known approach for scan detection [7], with a supervised model as a fourth layer. This allows us to empirically test fixed thresholds (Design 1) against supervised window learning on the exact same traffic, and to run both together, providing an evidence-based recommendation.

Evidence from Prototype Testing

The decision to establish the Hybrid IDS as the Primary Design is backed by our early prototype testing. In our controlled lab environment, the Suricata rules and our statistical layer detected simulated ping requests, nmap port scans, and hping3 SYN floods, and the dashboard showed these alerts almost immediately after we shortened the Suricata flow timeout. This hybrid needs no massive, labelled dataset, so it is the most reliable immediate solution. The Isolation Forest layer, tested on the Friday DDoS and PortScan files of CIC-IDS2017 [2] (used only as a benchmark), flagged few normal flows but detected none of the attack flows.

We have also built a first prototype of the Random Forest layer (10-second windows per source IP, trained on our own labelled Kali attacks). We compared it with each layer of Design 1 on the same 20,685 windows, of which 96 were attack windows (port scans of seven kinds, SYN floods of seven speeds, one UDP scan and one UDP flood). The Isolation Forest layer found none of them. The statistical layer found 90 of 96 with 6 false-alarm windows, and the Random Forest found 86 of 96 with 1. The two missed different attacks: the statistical layer missed the slowest SYN flood, which the Random Forest mostly found, while the Random Forest missed full-speed floods and UDP attacks that the statistical layer caught. Used together, they found 95 of the 96 attack windows with the same 6 false-alarm windows as the statistical layer alone. This supports adding the Random Forest as a fourth layer next to the existing ones rather than replacing any of them. The Rule and Isolation Forest layers alert on many normal windows, so the dashboard shows each layer as its own source that can be filtered.



Design	Attacks caught	False alarm window	Precision	Recall	F1
Design 1 Isolation Forest	0 / 96	298	0.00	0.00	0.00
Design 1 Rule layer	82 / 96	310	0.21	0.85	0.34
Design 1 Statistical layer	90 / 96	6	0.94	0.94	0.94
Design 2 Random Forest	86 / 96	1	0.99	0.90	0.94
Statistical layer + Random Forest	**95 / 96**	**6**	**0.94**	**0.99**	**0.96**

Skills Within the Group and Workload Distribution

Executing two parallel designs is possible because we divide the work based on what each team member already built during early prototyping. Moving into Design 2, this clear division of roles ensures no single member is overloaded while we add and evaluate the fourth detection layer alongside the existing three layers.

Team Member	Core Technical Area	Primary Design (3-Layer Hybrid)	Secondary Design (+4th Random Forest Layer)
Yan Han WONG	Capture Pipeline & Suricata	Sets up Suricata, manages multi-threaded eve.json output, and built the first working prototype.	Keeps the Suricata capture engine running to feed live flow records simultaneously to both designs.
Bruce Gee Yick WONG	ML Training & Evaluation	Trains the Isolation Forrest model on flow records; tests detection accuracy and alert latency.	Leads training and testing of the supervised Random Forrest layer using lab data and the CIC-IDS2017 dataset.
Neville Zi Yang	Dashboard & Alerting UI	Develops the real-time Flask-SocketIO dashboard, adds CSV log export, and coordinates the sprint schedule.	Updates the WebSocket dashboard to display Random Forest alerts as a fourth source next to the three Design 1 layers.
Jane Yan Zhen YU	Coordination & Documentation	Tracks deadlines, reviews the report, and research current domain techniques.	Manages final documentation and compares test metrics across individual and combined layers.

Timeline, Milestones, and Risks

The timelines for both designs run concurrently from Week 7 to Week 12, governed by a critical **Week 8 Decision Gate**. Because Design 2 only adds a fourth layer on top of Design 1 without altering the underlying capture pipeline, our schedule and technical risks remain strictly controlled.

Concurrent Execution & Decision Gate Summary (Weeks 7–12)

Week	Primary Design (3 Layers)	Secondary Design (+4th RF Layer)	Deliverable
Week 7 (19 Oct)	Merge 3-layer hybrid detection into main; finish dashboard redesign.	Collect labelled Kali attack data; train initial Random Forest model.	Working main branch & initial RF prototype.
Week 8 (26 Oct)	Test false alarms on normal browsing; tune statistical limits; re-check CIC-IDS2017.	Tune model; evaluate on lab data; verify CIC-IDS2017 TrafficLabelling files.	**Decision Gate:** Confirm whether to keep the 4th RF layer or switch it off.
Weeks 9–10	Run live Kali VM attack tests; draft evaluation report; clean GitHub repo.	Integrate RF alerts as a 4th dashboard source; finalize benchmark comparisons.	Stand-up 2 (2 Nov), unified 4-source UI, and report draft.
Weeks 11–12	Execute live attack demo; submit final report and repository.	Showcase 4-layer detection in live demo (if kept at Week 8) and document results.	Live demo, final evaluation report, GitHub repo, and peer assessment.

Cross-Design Risk and Fallback Summary

Core Risk	Affected Layer	Impact	Cross-Layer Mitigation & Fallback Strategy
Single-flow ML misses volumetric scans/floods	Design 1: Isolation Forest [1]	High	**Mitigated by 4th Layer:** The Random Forest layer groups traffic into 10s per-IP windows to learn scan and flood patterns directly from labelled data without manual thresholds.
Lab overfitting & failure on unseen attacks	Design 2: Random Forest [6]	High	**Mitigated by Design 1 Layers:** Supervised models often struggle with untrained traffic [3] or overfit to lab tools; keeping Design 1's rules, statistical layer, and unsupervised ML active catches unknown outliers.
15-second window delay on RF alerts	Design 2: Random Forest [6]	Low	**Mitigated by Parallel Alerting:** Design 1's rule and statistical alerts still fire immediately (<500 ms target), while the RF layer acts as a high-precision second opinion.
4th layer underperforms or lags schedule	Design 2: Random Forest [6]	Medium	**Zero-Disruption Fallback:** Because Design 2 is purely additive, we simply switch the RF layer off at Week 8

			and deliver Design 1 as a complete, working 3-layer system.

References

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422.

[2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[3] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security and Privacy*, Oakland, CA, USA, 2010, pp. 305–316.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[5] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Computational Intelligence for Security and Defense Applications (CISDA)*, Ottawa, ON, Canada, 2009, pp. 1–6.

[6] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[7] J. Jung, V. Paxson, A. W. Berger, and H. Balakrishnan, "Fast portscan detection using sequential hypothesis testing," in *Proc. IEEE Symp. Security and Privacy*, 2004, pp. 211–225.

Appendix A: Contribution of Each Team Member

Each section below is to be completed with the name of the team member who wrote it.

Section	Written by
Executive Summary	Neville
Primary Design: Design Concept	Neville
Primary Design: Methodology	Jane
Primary Design: Specification	Bruce
Primary Design: Milestones and Risks	Yan Han
Secondary Design: Design Concept	Neville
Secondary Design: Methodology	Jane
Secondary Design: Specification	Bruce
Secondary Design: Milestones and Risks	Yan Han
Design Justification	Yan Han
References and formatting	Jane